

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Bases de Datos I · Grupo 811

SolarDB Pascual: gobernanza de datos, automatización ETL e IoT

Trabajo de consulta en parejas

Docente: Ramiro Grisales Montoya

Fecha: 4 de octubre de 2026

Integrante 1: Erick Noel Delgado Serna

Integrante 2: Daniel Felipe Chica Giraldo

Documento preparado con base en la guía oficial suministrada para el trabajo. La evidencia de ejecución de PostgreSQL y GitHub debe corresponder a ejecuciones reales.

PARTE A · CONSULTA

G1. Definan data governance y diferencienla de data management y de data quality.

En SolarDB, la **gobernanza de datos** establece quién puede decidir sobre los datos, qué reglas deben cumplirse y quién responde por ellos. DAMA distingue esta función de la gestión de datos: *data management* reúne las actividades y capacidades para adquirir, almacenar, integrar, proteger y utilizar los datos. La *data quality*, por su parte, evalúa si los datos cumplen criterios como exactitud, completitud, consistencia y oportunidad. En SolarDB, la gobernanza definiría responsables y políticas; la gestión implementaría el almacenamiento, ETL y seguridad; y la calidad verificaría que la telemetría sea válida y útil. Por ejemplo, una regla que exige irradiancia dentro de un rango es una medida de calidad, mientras que asignar un responsable para aprobar esa regla pertenece a la gobernanza.

G2. Expliquen los roles data owner, data steward y data custodian. Propongan quién ocuparía cada rol en SolarDB y justifiquen.

El **data owner** responde por las decisiones sobre un dominio de datos y aprueba reglas de acceso y uso. El **data steward** aplica y supervisa las reglas de calidad, metadatos y definiciones del dato. El **data custodian** se ocupa de los aspectos técnicos de almacenamiento, seguridad, copias y operación. Para SolarDB proponemos que el responsable académico o líder del proyecto actúe como data owner; un integrante designado del equipo como data steward; y el responsable técnico de PostgreSQL/infraestructura como data custodian. Esta separación evita que una sola función concentre decisiones, calidad y operación técnica. Así, quien administra PostgreSQL no necesariamente decide qué significa una variable de la planta ni quién puede utilizarla.

G3. ¿Qué son los metadatos y qué es un catálogo de datos? ¿Qué columnas faltan al diccionario de datos?

Los **metadatos** son datos que describen otros datos: significado, tipo, origen, responsable, frecuencia, reglas y contexto. Un **catálogo de datos** organiza esos metadatos para que las personas puedan descubrir y comprender los activos de información. El diccionario de datos de SolarDB puede convertirse en un catálogo gobernado si, además de nombre de tabla, columna y tipo, incorpora como mínimo: **responsable**, **origen**, **regla de calidad** y **fecha de actualización**, que son campos solicitados explícitamente en la actividad. También sería útil registrar sensibilidad, periodicidad y descripción funcional. Esto permite conocer no solo cómo se almacena una variable, sino quién responde por ella, de dónde proviene y bajo qué condiciones debe considerarse válida.

G4. Describan las dimensiones de calidad de datos y traduzcan cuatro a restricciones PostgreSQL.

Las dimensiones son **completitud** (no faltan datos necesarios), **validez** (cumplen reglas permitidas), **unicidad** (no existen duplicados indebidos), **consistencia** (no hay contradicciones), **oportunidad** (llegan a tiempo) y **exactitud** (representan correctamente el fenómeno). En SolarDB se pueden convertir cuatro dimensiones en restricciones: **NOT NULL** para exigir dispositivo y fecha, asociada a completitud; **CHECK** para limitar p_ac e irradiancia, asociada a validez; **UNIQUE** o una clave primaria para evitar lecturas repetidas, asociada a unicidad; y **FOREIGN KEY** para asegurar que una lectura corresponda a un dispositivo existente, asociada a consistencia referencial. Las restricciones convierten reglas conceptuales en controles ejecutables por PostgreSQL.

G5. Expliquen el principio de mínimo privilegio y diseñen tres roles para SolarDB.

El **principio de mínimo privilegio** indica que cada usuario o proceso debe recibir únicamente los permisos necesarios para cumplir su función. En SolarDB proponemos tres roles. **solar_ingesta**: puede conectarse y ejecutar las operaciones necesarias sobre staging y carga, sin permisos administrativos. **solar_consulta**: solo puede realizar SELECT sobre las tablas destinadas a consulta. **solar_admin**: administra tablas, roles y configuraciones, reservado al responsable técnico. La práctica implementa específicamente **solar_lector** con SELECT sobre lectura_demo y sin INSERT, UPDATE ni DELETE. Esta separación reduce el impacto de credenciales comprometidas y facilita auditoría, porque cada acción puede asociarse con una función concreta.

G6. Ley 1581 de 2012: ¿la telemetría de la planta es un dato personal? ¿Qué cambia si se registran operadores?

La telemetría técnica de una planta —por ejemplo potencia, irradiancia o temperatura— no es por sí misma un dato personal si no identifica o hace identificable a una persona natural. La situación cambia si SolarDB almacena datos de operadores, porque información asociada a una persona identificada o identificable puede quedar dentro del ámbito de protección de datos personales. Nuestra postura ética es aplicar minimización: almacenar únicamente los datos necesarios, limitar accesos, definir finalidades, proteger credenciales y evitar reutilizar información para propósitos

incompatibles. Si se registran operadores, deben establecerse controles adicionales de acceso, trazabilidad y tratamiento responsable. La evaluación jurídica concreta depende del contexto y de si el dato permite identificar directa o indirectamente a una persona.

E1. Diferencien ETL de ELT y ubiquen SolarDB.

En **ETL** los datos se extraen, transforman y después se cargan al destino. En **ELT** se extraen y cargan primero, dejando la transformación para ejecutarse dentro de la plataforma de destino. El flujo propuesto para SolarDB se parece más a **ELT**: el mensaje crudo del gateway se conserva en la columna **payload JSONB** de staging y posteriormente SQL extrae sus atributos para llevarlos a la tabla relacional. Esto tiene una ventaja de trazabilidad porque se conserva el mensaje original. PostgreSQL ofrece operadores nativos para consultar JSON/JSONB, por lo que la transformación puede realizarse cerca del almacenamiento. El proceso sigue siendo una canalización ETL/ELT automatizada porque existe extracción, carga, transformación y registro del resultado.

E2. Comparen carga completa e incremental. ¿Qué significa idempotencia y cómo ayuda ON CONFLICT?

Una **carga completa** vuelve a procesar todo el conjunto de datos; una **carga incremental** procesa únicamente datos nuevos o modificados. Una operación es **idempotente** cuando repetirla con la misma entrada produce el mismo estado final, sin duplicar efectos. En SolarDB, la clave primaria de **lectura_demo** es la pareja (**dispositivo_id, ts**). Por tanto, si el mismo mensaje se procesa dos veces, la segunda inserción encuentra el conflicto. La cláusula **ON CONFLICT (dispositivo_id, ts) DO NOTHING** evita crear una segunda fila. Esto hace que repetir el pipeline sea seguro y facilita reintentos después de errores. La prueba solicitada consiste en ejecutar dos veces el mismo archivo y comparar el COUNT: el segundo ciclo no debe aumentar la cantidad de lecturas válidas.

E3. Comparen cron, pg_cron y Apache Airflow y elijan uno.

cron es un planificador del sistema operativo que ejecuta comandos según una expresión temporal; es simple y apropiado para una práctica pequeña. **pg_cron** permite programar tareas desde PostgreSQL y resulta cómodo cuando el trabajo está estrechamente ligado a la base. **Apache Airflow** es una plataforma de orquestación de workflows que permite definir, programar y monitorear pipelines como código. Para esta primera fase de SolarDB elegiríamos **cron**, porque el flujo tiene un único punto de entrada y una periodicidad sencilla. Si el proyecto creciera a múltiples dependencias, reintentos, tareas y fuentes, Airflow sería más apropiado. La elección debe considerar la complejidad operativa y no solo la capacidad técnica del programador.

E4. ¿Qué debe registrar la bitácora ETL y por qué es gobernanza?

La bitácora debe registrar como mínimo **inicio, fin, filas leídas, filas cargadas, filas rechazadas, estado y error**. Para SolarDB, estos campos permiten saber cuándo se ejecutó el proceso, cuánto procesó y si hubo fallas. También permiten comparar ejecuciones y detectar anomalías, como una caída inesperada en el número de lecturas. La bitácora es una herramienta de gobernanza porque aporta **trazabilidad**: permite relacionar una carga con su ejecución y conservar evidencia de lo que ocurrió. Esa trazabilidad contribuye al concepto de *data lineage*, ya que ayuda a seguir el recorrido del dato desde el archivo o mensaje hasta la tabla relacional. En caso de incidente, facilita investigar qué proceso produjo un resultado y qué registros fueron afectados.

I1. Describan la arquitectura IoT por capas y ubiquen los elementos.

La arquitectura propuesta tiene cinco capas: **dispositivo**, donde sensores e inversores producen mediciones; **gateway/edge**, donde se reciben, normalizan o filtran mensajes; **red/broker**, que transporta y distribuye mensajes; **almacenamiento**, donde SolarDB conserva staging y datos relacionales; y **aplicación**, que consume la información. En SolarDB, el inversor y el medidor pertenecen a la capa de dispositivo; el piranómetro también produce una medición física. El gateway concentra las lecturas y puede publicirlas por MQTT. El broker representa la capa de comunicación. PostgreSQL corresponde al almacenamiento. La aplicación que consulta las lecturas pertenece a la última capa. Esta separación permite controlar responsabilidades y facilita escalar la ingesta sin acoplar directamente los dispositivos con la base.

I2. Expliquen MQTT y propongan topics.

MQTT utiliza un modelo **publish/subscribe**: los clientes publican mensajes en un **topic** y otros clientes se suscriben a filtros de topic a través de un **broker**. El protocolo define tres niveles de QoS: **0**, como máximo una entrega; **1**, al menos una entrega; y **2**, exactamente una entrega. Para SolarDB proponemos topics jerárquicos como **pascualbravo/solar/sede/inversor-01/p_ac** y **pascualbravo/solar/sede/inversor-01/irradiancia**. Esta organización facilita suscripciones por planta, dispositivo o variable. La elección del QoS debe depender del costo de perder o repetir mensajes: para telemetría frecuente puede bastar QoS 0 o 1, mientras que un evento que requiera mayor garantía

puede justificar QoS 2.

I3. Comparen Modbus RTU/TCP, MQTT y OPC UA.

Modbus RTU es un protocolo industrial serial basado normalmente en maestro/esclavo o cliente/servidor, frecuente en dispositivos y medidores. **Modbus TCP** lleva el modelo Modbus sobre redes TCP/IP. **MQTT** usa publish/subscribe mediante un broker y transporta un payload cuyo formato puede ser JSON u otro formato definido por la aplicación.

OPC UA es una arquitectura de interoperabilidad industrial con un modelo de información más rico y mecanismos de seguridad. En SolarDB, Modbus puede servir para adquirir datos del inversor, MQTT para transportar telemetría desde el gateway y OPC UA para integración industrial cuando se necesite interoperabilidad semántica y seguridad más completa. La elección depende del equipo existente, latencia, arquitectura y requisitos de seguridad.

I4. Diseñen un payload JSON de inversor y dos consultas JSONB.

Un payload válido para la práctica puede contener: **device_id**, **ts**, **p_ac** en kW, **irradiancia** en W/m², **temp_modulo** en °C y **voltaje_dc** en V. El campo nuevo elegido para esta implementación es **voltaje_dc**. PostgreSQL permite extraer valores JSONB con operadores nativos. Por ejemplo, **payload ->> 'device_id'** obtiene el valor como texto y luego puede convertirse a entero. Otra consulta puede usar **payload ? 'alarma'** para identificar mensajes que contienen la clave alarma. Una consulta adicional podría utilizar **@>** para comprobar que un JSON contiene un fragmento determinado. Esto permite conservar el mensaje original y, al mismo tiempo, convertir atributos relevantes a columnas relacionales.

I5. Estimen filas por mes y expliquen particionamiento, BRIN y TimescaleDB.

Con 10 dispositivos, 5 variables y una lectura por minuto, hay $10 \times 5 \times 60 \times 24 \times 30 = 2.160.000$ valores de variable por mes si cada variable se modela como una fila. Si cada mensaje agrupa las cinco variables, serían 432.000 mensajes mensuales. El **particionamiento por rango** puede separar las filas por periodos de tiempo, por ejemplo meses, reduciendo el volumen que debe revisar una consulta temporal. Un índice **BRIN** es útil para tablas grandes cuando los valores indexados tienen correlación con el orden físico, como una columna temporal insertada aproximadamente en orden. **TimescaleDB** es una alternativa orientada a series temporales que puede simplificar algunas operaciones de este tipo. La decisión depende del volumen y del patrón real de consultas.

V1. Diferencien Git y GitHub y expliquen los conceptos solicitados.

Git es un sistema de control de versiones distribuido; **GitHub** es una plataforma que aloja repositorios Git y añade colaboración, revisión y gestión del proyecto. Un **repository** contiene el historial y archivos versionados. Un **commit** registra un cambio. Una **branch** permite trabajar en una línea de desarrollo. Un **pull request** propone integrar cambios y revisarlos. **.gitignore** define archivos que Git no debe incluir. Nunca se deben subir contraseñas o cadenas de conexión porque el historial puede conservarlas incluso después de borrar el archivo. Para SolarDB se debe usar un archivo **.env** excluido por **.gitignore** y variables de entorno, dejando únicamente un **.env.example** sin secretos. También conviene revisar los commits antes de hacer público el repositorio.

PARTE B · PRÁCTICA GUIADA

El código completo se encuentra en el repositorio. El PDF debe contener capturas reales de la ejecución. Para evitar presentar evidencia simulada, las áreas siguientes se dejan identificadas para insertar las capturas obtenidas al ejecutar PostgreSQL y GitHub.

B1 · Simulador IoT

Ejecutar ``python etl/simulador.py``. Captura requerida: terminal y/o archivo ``data/lecturas.jsonl`` mostrando mensajes de los dos dispositivos y el campo ``voltaje_dc``.

B2 · Staging

Captura requerida: ``SELECT COUNT(*) FROM stg_lectura_raw;`` y una consulta que muestre el ``payload`` JSONB cargado.

B3 · Transformación

Captura requerida: ejecución de ``sql/02_carga.sql`` y registros presentes en ``lectura_demo``.

B4 · Idempotencia

Capturas requeridas de ``SELECT COUNT(*) FROM lectura_demo;`` después de la primera y segunda ejecución con el mismo archivo. El segundo conteo debe permanecer igual.

B5 · Bitácora

Captura requerida: ``SELECT * FROM etl_log ORDER BY id DESC;`` mostrando inicio, fin, filas leídas, cargadas, rechazadas y estado.

B6 · Rol de solo lectura

Capturas requeridas: un `SELECT` exitoso con ``solar_lector`` y un `INSERT` rechazado con el mismo rol.

B7 · Cron

La configuración propuesta es ``0 * * * * cd /ruta/al/repositorio && /usr/bin/python3 etl/run_etl.py``. La guía indica que no es necesario implementarla.

DIAGRAMA DE ARQUITECTURA IoT

DISPOSITIVOS Inversor · Medidor · Piranómetro	GATEWAY / EDGE Normalización y envío
RED / BROKER MQTT · Topics	ALMACENAMIENTO PostgreSQL · staging · lectura_demo
APLICACIÓN Consultas y consumo de datos	

Flujo conceptual: los dispositivos producen telemetría; el gateway la concentra; el broker MQTT distribuye mensajes; PostgreSQL conserva el mensaje crudo y la representación relacional; la aplicación consulta los datos.

PARTE C · REPOSITORIO EN GITHUB

Repositorio: <https://github.com/>_____/solardb-apellido1-apellido2

La guía exige que el repositorio sea público, que ambos integrantes aparezcan como colaboradores y que cada integrante tenga mínimo tres commits descriptivos.

Evidencia GitHub pendiente de captura real:

1. Collaborators: deben aparecer ambos integrantes.
2. Historial de commits: deben verse autores de ambos integrantes.
3. README: debe mostrar nombres, curso, descripción, reproducción y aportes.

REFERENCIAS

1. DAMA International. What is Data Management? Referencia al marco DAMA-DMBOK y al papel de la gobernanza. <https://dama.org/about-dama/what-is-data-management/>
2. PostgreSQL Global Development Group. PostgreSQL Documentation — JSON Functions and Operators. <https://www.postgresql.org/docs/current/functions-json.html>
3. OASIS. MQTT Version 5.0 — OASIS Standard. <https://www.oasis-open.org/standard/mqtt-v5-0-os/>
4. Apache Software Foundation. Apache Airflow Documentation. <https://airflow.apache.org/docs/apache-airflow/stable/>
5. Colombia, Ley 1581 de 2012. Régimen general de protección de datos personales. Consultar la versión oficial en el portal normativo colombiano.
6. Guía oficial del trabajo de consulta SolarDB Pascual, Bases de Datos I, Grupo 811, 2026-II, docente Ramiro Grisales Montoya.

Nota de integridad académica: las respuestas y scripts son un borrador completo basado en la guía. Las capturas de PostgreSQL, GitHub, colaboradores y commits deben obtenerse de ejecuciones reales; no deben fabricarse.